Extension Methods

Add new methods to existing types in C#, without changing them

1 What is it?

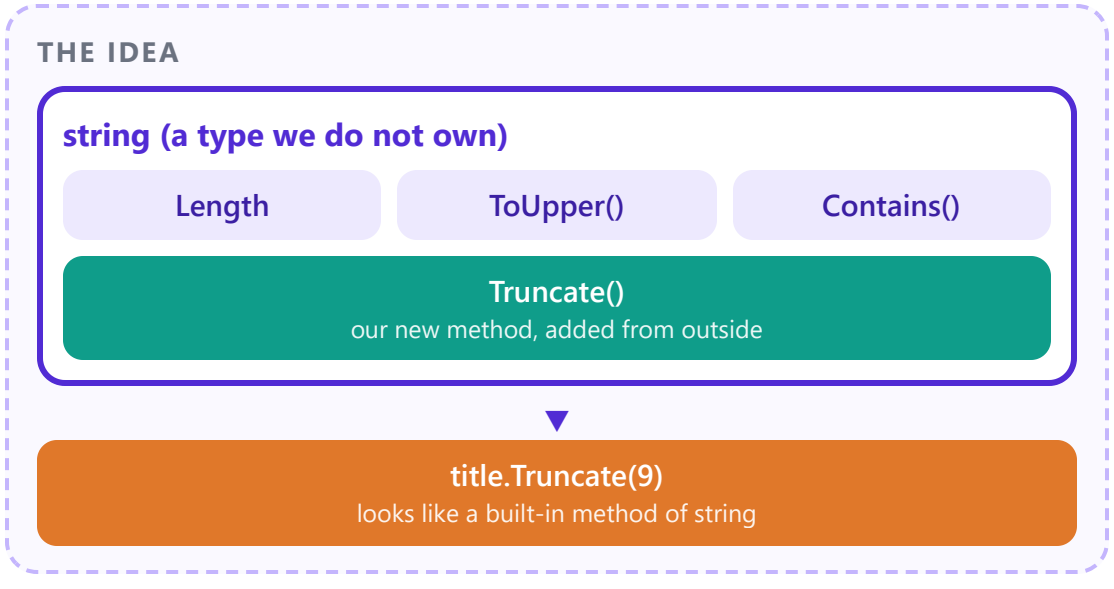
A way to **add a new method to an existing type** without modifying its code and without inheriting from it. You call it like a normal method of that type.

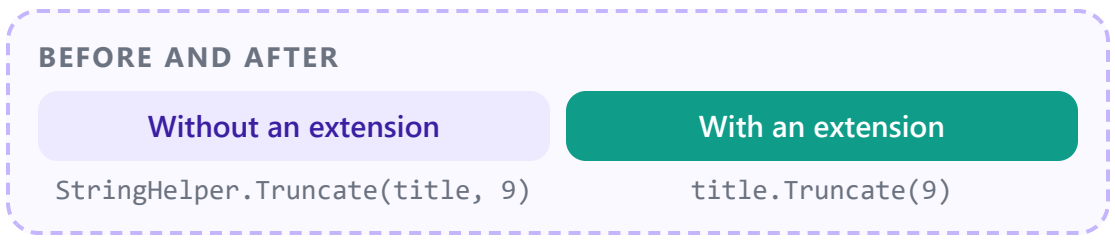
2 Why does it exist?

Sometimes we want to add a method to a type that:

- we **do not own**, like `string` or `List<T>`, which are part of .NET.
- we **cannot inherit** from, like `string` (it is sealed).

It also gives cleaner calls that can be **chained**.





The three rules

Every extension method follows the same shape

1 A static class

The method lives inside a **static class**.

2 A static method

The method itself is **static**.

3 The this keyword

The **first parameter** starts with `this`. Its type is the type we are extending.

> The first parameter is the object we call the method on. It is not written between the brackets when we call the method.

EXAMPLE

```
public static class StringExtensions
{
    public static string Truncate(
        this string text, int maxLength)
    {
        if (text.Length <= maxLength)
            return text;

        return text.Substring(0, maxLength) + "...";
    }
}
```

USING IT

```
string title = "Extension methods in C#";
Console.WriteLine(title.Truncate(9));   // Extension...
```

> Only **maxLength** is passed in the call. The object before the dot (**title**) is the **this** parameter.

How it works

An extension method is a normal static method, with a nicer way to call it

WHAT THE COMPILER DOES

`title.Truncate(9)`
what you write

▼ **the compiler rewrites it**

`StringExtensions.Truncate(title, 9)`
what really runs

The `this` keyword is just **syntactic sugar**. Behind the scenes it is a regular static method that receives the object as its first argument.

WHY THE USING MATTERS

`using MyProject.Extensions;`
title.Truncate(9) works

▼ **remove the using line**

Compile error CS1061
'string' does not contain a definition for 'Truncate'

WHAT FOLLOWS FROM THAT

1 No private access

It cannot access the **private** members of the type. It is not part of the type.

2 The real method wins

If the type already has a method with the **same name and parameters**, the real method runs and the extension is ignored.

3 Import the namespace

The method is visible only if the namespace of its class is imported with `using`.

You already use them

LINQ is built entirely from extension methods

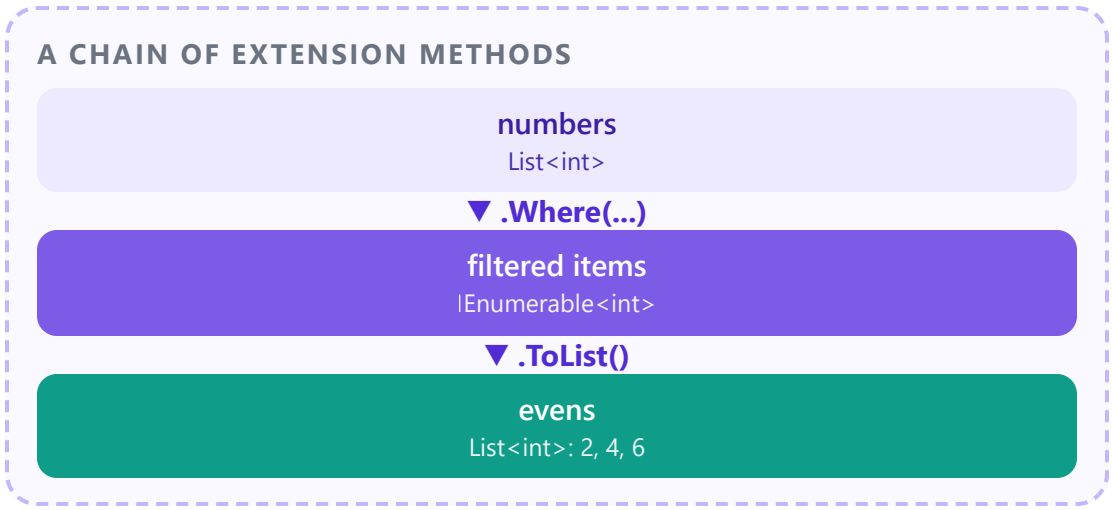
LINQ

`Where`, `Select`, `ToList`, `OrderBy` and `FirstOrDefault` are not members of `List<T>`. They are **extension methods** added to `IEnumerable<T>`.

```
List<int> numbers = new List<int> { 1, 2, 3, 4, 5, 6 };

List<int> evens = numbers
    .Where(n => n.IsEven())
    .ToList();
```

> `Where` and `ToList` come from .NET. `IsEven` is our own extension on `int`. They work together because they are all extension methods.



Writing your own

More examples, and why chaining works

EXAMPLES

```
public static class StringExtensions
{
    public static string Capitalize(this string text)
    {
        if (string.IsNullOrEmpty(text))
            return text;

        return char.ToUpper(text[0]) + text.Substring(1);
    }
}

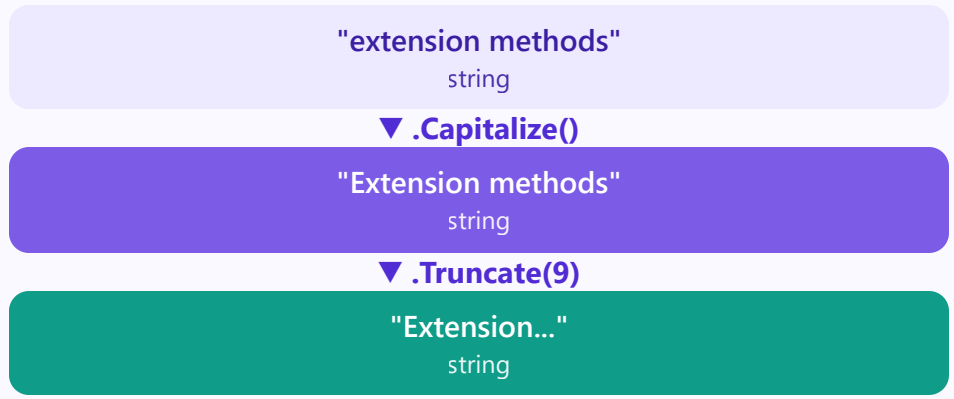
public static class IntExtensions
{
    public static bool IsEven(this int number)
    {
        return number % 2 == 0;
    }
}
```

USING THEM

```
Console.WriteLine(4.IsEven());       // True
Console.WriteLine(7.IsEven());       // False

string result = "extension methods".Capitalize().Truncate(9);
Console.WriteLine(result);          // Extension...
```

WHY CHAINING WORKS



Each method **returns the same type** it extends, so the next method can be called on the result.

Where to put them, and when to use them

Organisation, good uses, and a summary

WHERE TO PUT THE FILES

```
MyProject/
├── Extensions/
│   ├── StringExtensions.cs
│   └── IntExtensions.cs
└── Program.cs
```

- A folder named **Extensions**.
- The file and class names end with **Extensions**.
- One file for each extended type, so you know where to look.

USE THEM WHEN

- You want to add a useful method to a type you do not own.
- You want clean, chainable calls.

> **Avoid them** when the type is yours (add the method inside it), or when you add so many that nobody knows where a method comes from.

SUMMARY

Concept	What it is
Extension method	A method added to an existing type without changing its code
The rules	`static class` + `static` method + `this` on the first parameter
Under the hood	A normal static method, rewritten by the compiler
Limits	No private access, the real method wins, needs the `using`

TRY IT YOURSELF

- Write an extension on `string` called `IsEmail` that checks the text contains `@` and a dot.
- Write `IsOdd` on `int` and use it with `Where`.
- Remove the `using` line and read the error.